Software Requirements Specification

For

<Vortex>

<March 14, 2026>

Prepared by

Specialization	SAP ID	Name
Graphics and Gaming	500119378	Keerti Vardhan
Data Science	500119425	Yuvraj Siyaar
Graphics and Gaming	500120636	Ayushmaan Kapruwan
Graphics and Gaming	500120527	Aditya Kediyal

School Of Computer Science

UNIVERSITY OF PETROLEUM & ENERGY STUDIES
DEHRADUN – 248007, Uttarakhand

CONTENTS

1 INTRODUCTION **3**
1.1 Purpose of the Project . . . 3
1.2 Target Beneficiary . . . 3
1.3 Project Scope . . . 4
1.4 References . . . 4

2 PROJECT DESCRIPTION **5**
2.1 Reference Algorithm . . . 5
2.2 Characteristic of Data . . . 5
2.3 SWOT Analysis . . . 7
2.4 Project Features . . . 8
2.5 User Classes and Characteristics . . . 10
2.6 Design and Implementation Constraints . . . 10
2.7 Design Diagrams . . . 11
2.8 Assumption and Dependencies . . . 18

3 SYSTEM REQUIREMENTS **18**
3.1 User Interface . . . 18
3.2 Software Interface . . . 19
3.3 Database Interface . . . 19
3.4 Protocols . . . 19

4 NON-FUNCTIONAL REQUIREMENTS **20**
4.1 Performance Requirements . . . 20
4.2 Security Requirements . . . 20
4.3 Software Quality Attributes . . . 20

5 OTHER REQUIREMENTS **21**
5.1 Evaluation-Oriented Requirements . . . 21
5.2 Testing Requirements . . . 21
5.3 Documentation Requirements . . . 21

A Glossary **22**

B Analysis Model **22**

C Issues List **23**

LIST OF FIGURES

1 Use Case Diagram Level-2 (Source: By Team) 9
2 Use Case Diagram (Source: By Team) 11
3 Activity Diagram (Source: By Team) 12
4 Class Diagram 2 (Source: By Team) 13
5 Sequence Diagram (Source: By Team) 14
6 Data Flow Diagram (Source: By Team) 15
7 Data Flow Diagram Level-0 (Source: By Team) 16
8 State Diagram (Source: By Team) 17

REVISION HISTORY

Date	**Change**	**Reason for Changes**	**Mentor Signature**

1. INTRODUCTION

Vortex is a unified game library management application with an intelligent recommendation system. The app detects games from multiple platforms into a single interface & also tracks playtime, session patterns, and user preferences.

Built using C++ and Qt/QML for performance, Vortex implements a machine learning-based recommendation engine that uses content-based filtering and mood-aware personalization. The system analyses user behavior like play duration, time-of-day patterns, and explicit ratings to generate relevant game suggestions.

A modular architecture allows users to customize features according to their needs, avoiding the bloat common in existing launchers. The project demonstrates practical application of recommender systems, data pipeline design, and native software development.

Keywords: Game Library Management, Recommender Systems, Mood-Aware, Personalization, Desktop Application, Machine Learning

1.1 Purpose of the Project

Digital game distribution has evolved significantly over the past decade. Users now own games across Steam, Epic Games Store, GOG, Xbox Game Pass, and other platforms. Each platform maintains its own launcher, library interface, and usage statistics, creating a fragmented experience for gamers who must switch between multiple applications.

Current game launchers operate in isolation, offering no unified view of a user's complete game collection. Playtime data remains isolated, making it impossible to analyze overall gaming habits. Existing recommendation systems rely primarily on purchase history and popularity metrics rather than actual user behavior and contextual factors like mood or available time.

1.2 Target Beneficiary

Target users include:

- Gamers who own games across multiple digital distribution platforms.
- Users who want a centralized game library instead of switching between launchers.
- Players interested in tracking gameplay patterns such as playtime and session frequency.

- Users who want personalized game recommendations based on mood, time, or previous activity.

1.3 Project Scope

The scope of the Vortex system includes:

- Detecting installed games from multiple digital platforms.
- Aggregating these games into a unified interface.
- Tracking gameplay statistics such as playtime and session history.
- Providing a recommendation system based on user behavior.
- Offering a modular architecture that allows optional features.

The system does not aim to:

- Replace existing game launchers.
- Modify or distribute game files.

1.4 References

REFERENCES

GOG Developer Documentation: Quick Start (2026). URL: https://docs.gog.com/quick-start/ (visited on 03/14/2026).

IGDB API Documentation (2026). URL: https://www.igdb.com/api (visited on 03/14/2026).

Kozakov, M. and N. Kozakova (2025). "Development of a Recommendation System for Video Games." In.

Playnite Documentation (2026). URL: https://api.playnite.link/docs/manual/gettingStarted/gettingStartedOverview.html (visited on 03/14/2026).

Steamworks Documentation (2026). URL: https://partner.steamgames.com/doc/home (visited on 03/14/2026).

2. PROJECT DESCRIPTION

2.1 Reference Algorithm

Vortex recommendation module follows a hybrid content-driven approach:

- **Content Similarity**
  - TF-IDF vectorization performed over game metadata (genres, themes, and descriptions).
  - Cosine similarity calculation between candidate games and user-liked titles.
- **Preference Adjustment**
  - Conversion of binary feedback (Like/Dislike) into a weighted preference score.
- **Mood Adjustment**
  - Encoding of user-selected moods using one-hot vectors.
  - Application of a mood-genre mapping to modify candidate scores dynamically.
- **Final Rank**
  - Weighted aggregation of similarity, preference, quality, and an exploration factor.

Note: The current Python implementation is a work-in-progress with known suggestion inaccuracies; calibration and validation are ongoing.

2.2 Characteristic of Data

Vortex utilizes a combination of local behavioral data and external game metadata to power its recommendation engine and library management features. The dataset is characterized by a mix of categorical, numerical, text-based, and temporal information.

Primary and Secondary Sources

Primary Data Sources The primary dataset is generated locally through user interactions with the launcher. This includes detected local game states (Game Name, Version, Install Directory) and continuously produced behavioral metrics (Session Start/End times, Idle/Inactive duration, total Session length, Mood Session IDs, and explicit Liked/Disliked Games).

Secondary Data Sources The secondary dataset is sourced externally from the Internet Game Database (IGDB) and digital distribution APIs. IGDB provides rich, structured JSON metadata via a REST API, contributing Genre/Tags, Game Length, Developer/Publisher details, and Ratings/Reviews. Additional platform-specific data, such as in-game achievements, is retrieved from APIs like Steam's.

Sampling Techniques Data collection relies on real-time event capture rather than traditional batch data sampling. User actions and session logs are continuously recorded as raw events and stored dynamically in the local SQLite database. External metadata is not sampled from a static database dump; instead, it is fetched dynamically via HTTP POST requests to the IGDB API as needed, strictly governed by a rate limit of 4 requests per second.

Statistical Methods and Data Processing The system employs several statistical and machine learning techniques to process and quantify raw data for the recommendation pipeline:

One-Hot Encoding Applied to convert categorical data—such as game genres and user moods (e.g., "Relaxed", "Competitive")—into a binary numerical format for model processing.

TF–IDF Vectorization (Term Frequency – Inverse Document Frequency) is used to process unstructured text documents, such as game descriptions, converting them into multi-dimensional numerical float vectors.

Normalization Continuous variables, such as total playtime duration and external review ratings (e.g., Metacritic scores), are scaled and normalized to float values between 0.0 and 1.0 to ensure uniform weight distribution in the ranking algorithm.

Cosine Similarity Utilized within the content-based filter to mathematically determine the distance and similarity between the TF–IDF feature vectors of candidate games and those the user has positively rated.

Weighted Aggregation The Hybrid Ranker calculates a final recommendation score by applying mathematical weights to various quantified inputs: Preference Match (0.40), Similar to Liked Games (0.35), Quality Score (0.15), and an Exploration Factor (0.10) to output a final percentage-based rank.

2.3 SWOT Analysis

- **Strengths**
  - Unified launcher concept providing practical utility for game management.
  - High-performance execution leveraging native C++.
  - Extendable architecture designed for future machine learning evolution.
- **Weaknesses**
  - Inconsistent recommendation quality in the current iteration.
  - Partial integration between the C++ core and Python modules.
  - User Interface is not yet finalized in its intended QML form.
- **Opportunities**
  - Integration of additional platforms such as GOG and Epic Games.
  - Enhancement of recommendation logic through feature engineering and model tuning.
  - Positioning as a lightweight alternative to resource-heavy game launchers.
- **Threats**
  - Potential API changes or rate limits from external services like Steam and IGDB.
  - Data sparsity issues, specifically regarding the ”cold start” problem for new users.
  - Risk of scope creep within the constraints of the academic timeline.

2.4 Project Features

- **Multi-source Game List**: Integration of both Steam and local directory scanning.
- **Direct Launch**: Ability to initialize games directly from the Vortex interface.
- **Heuristic Filtering**: Basic exclusion of non-game executables during the directory scanning process.
- **Interactive Navigation**: Sorting and selection capabilities via an interactive menu.
- **Planned Features**: Implementation of analytics dashboards and mood-based recommendation displays.

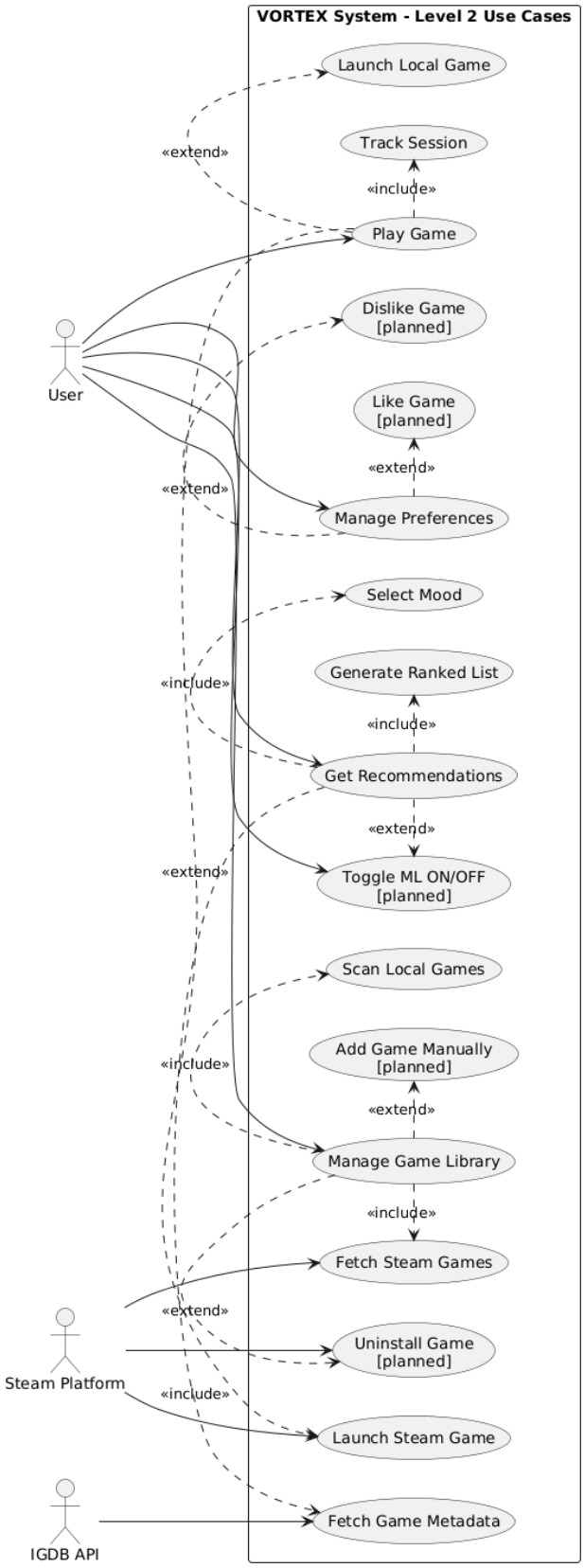

Figure 1: Use Case Diagram Level-2 (Source: By Team)

2.5 User Classes and Characteristics

- **End User (Gamer)**
  - Possesses basic technical knowledge.
  - Requires a simple interaction model and low-latency system responses.
- **Project Developer**
  - Responsible for core modules (C++, Python, Database, and UI).
  - Requires modular, decoupled, and testable components for iterative development.
- **Evaluator / Faculty**
  - Reviews the technical methodology, system architecture, and requirement traceability.
  - Validates the alignment between project goals and final implementation.

2.6 Design and Implementation Constraints

- **Primary OS Target**: System is specifically designed for the Windows operating system.
- **Steam Data Dependencies**: Retrieval relies on access to the Windows Registry and the local Steam file structure.
- **Local Discovery**: Local scanning logic currently assumes executable-based discovery targeting `.exe` files.
- **Recommendation Constraints**: Quality is contingent upon the completeness of metadata and the depth of available user history.
- **Temporal Constraints**: Project development is limited to a 12-week academic implementation window.

2.7 Design Diagrams

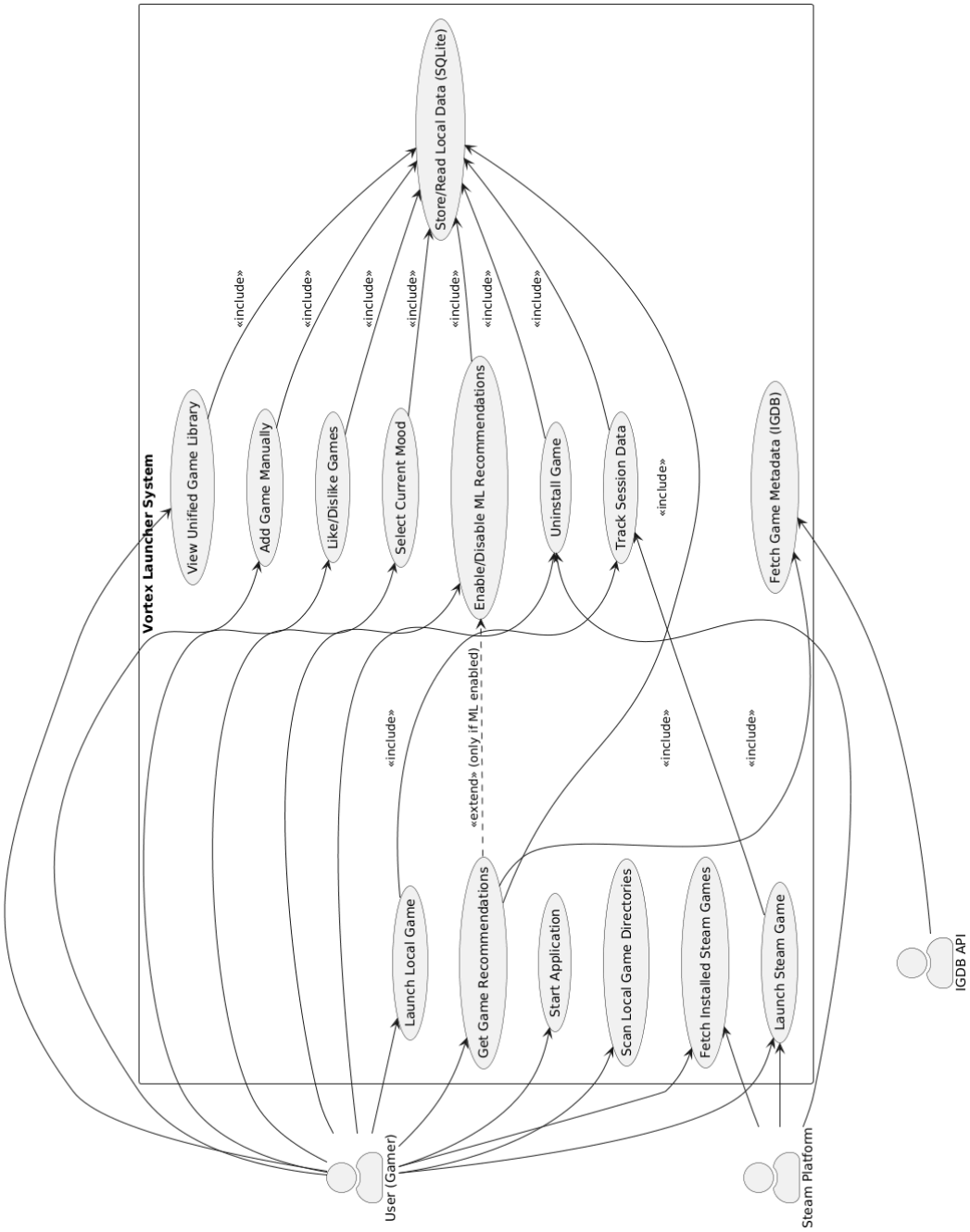

Figure 2: Use Case Diagram (Source: By Team)

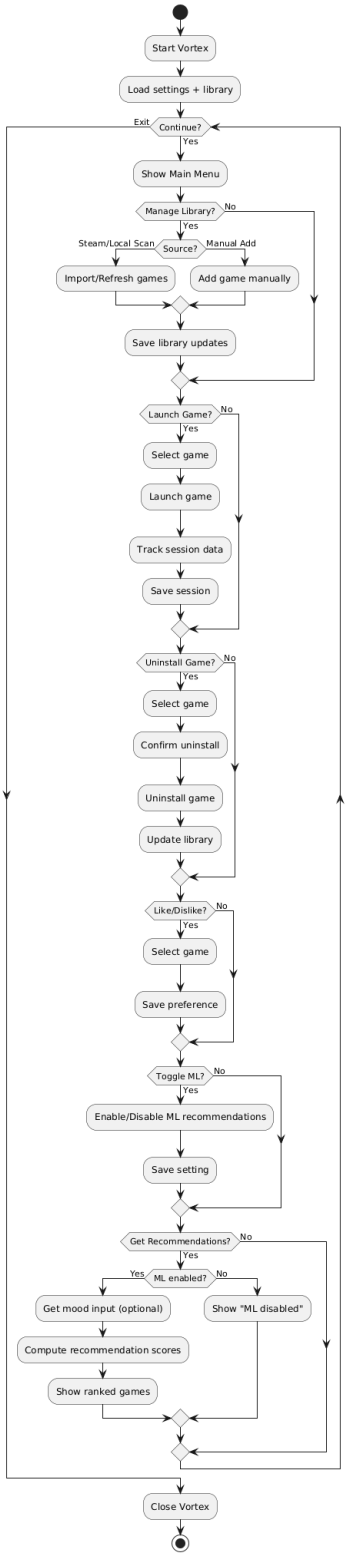

Figure 3: Activity Diagram (Source: By Team)

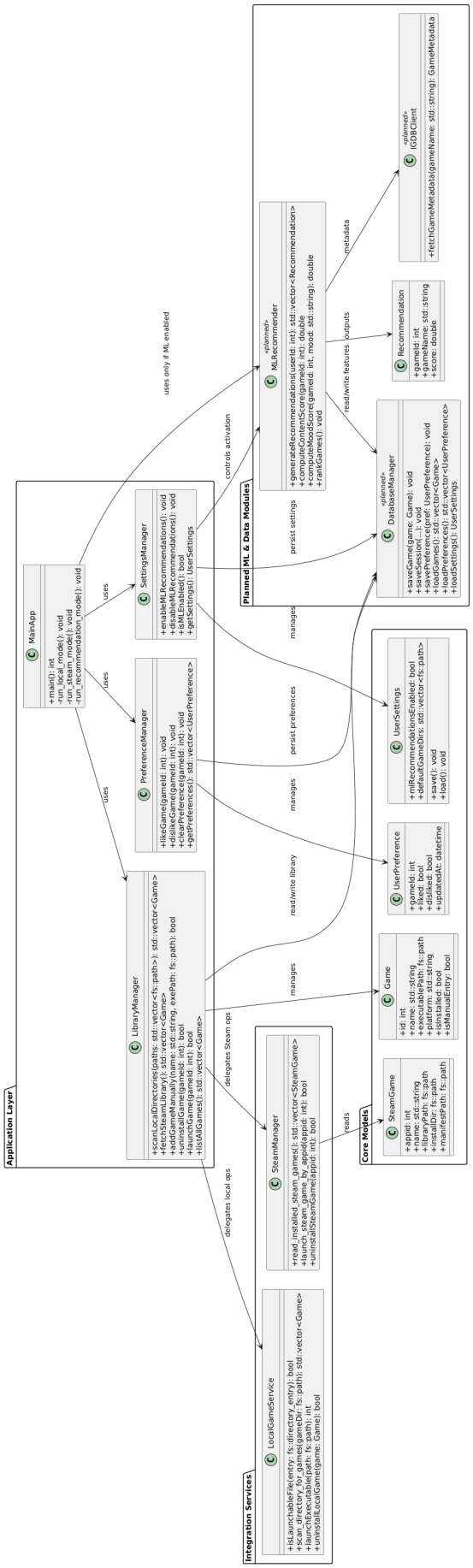

Figure 4: Class Diagram 2 (Source: By Team)

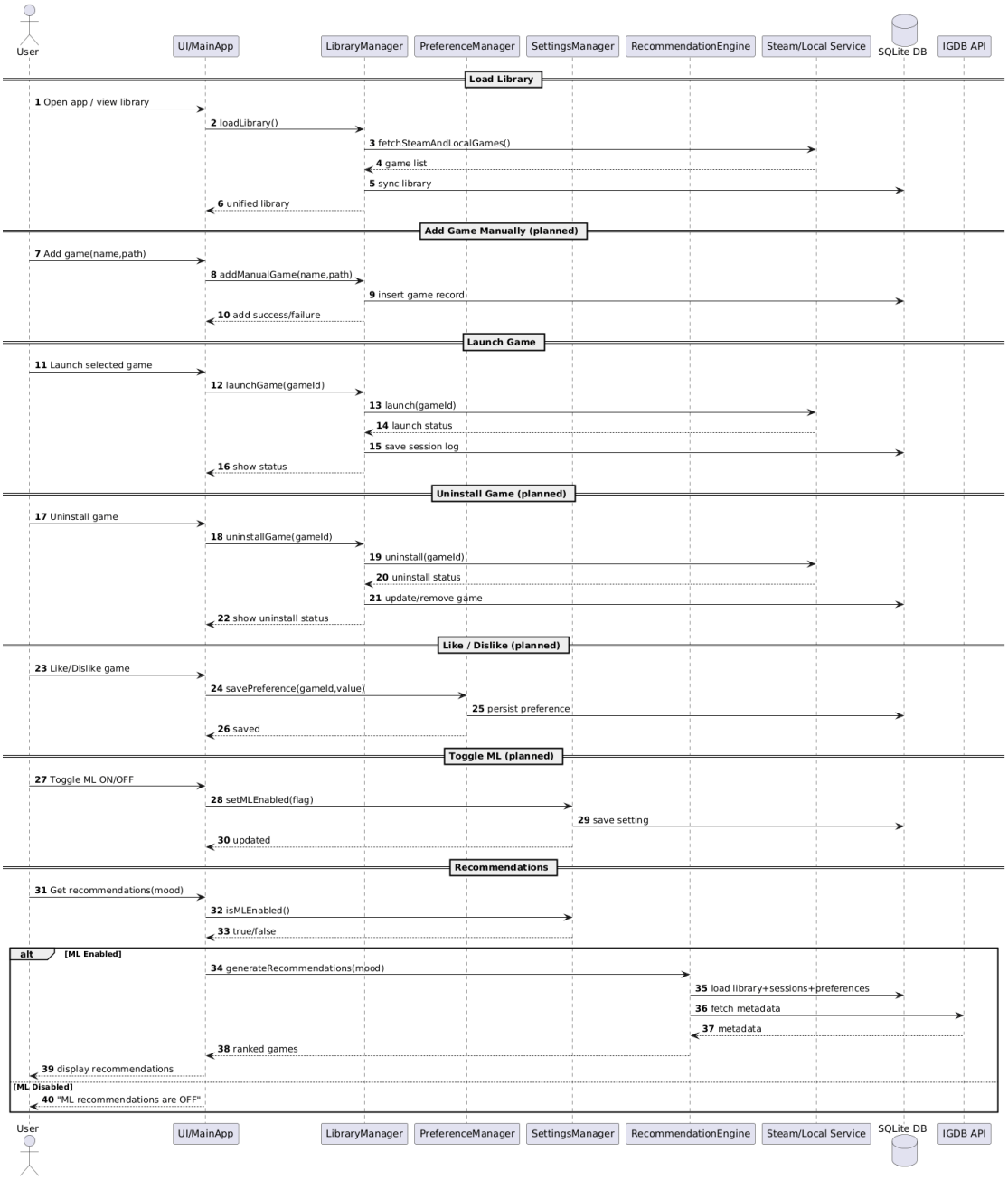

Figure 5: Sequence Diagram (Source: By Team)

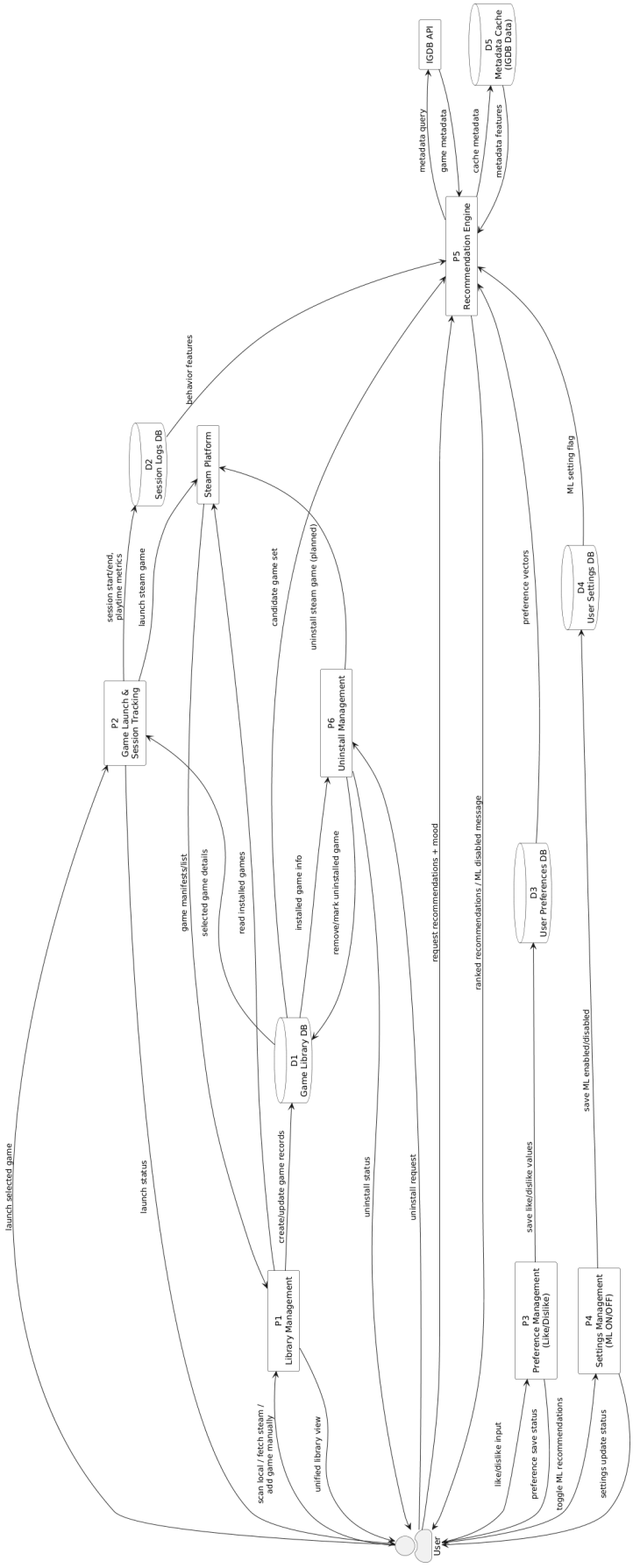

Figure 6: Data Flow Diagram (Source: By Team)

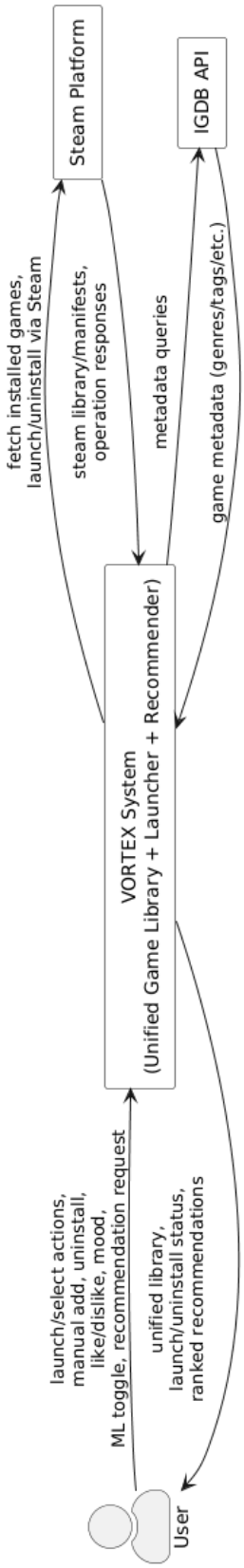

Figure 7: Data Flow Diagram Level-0 (Source: By Team)

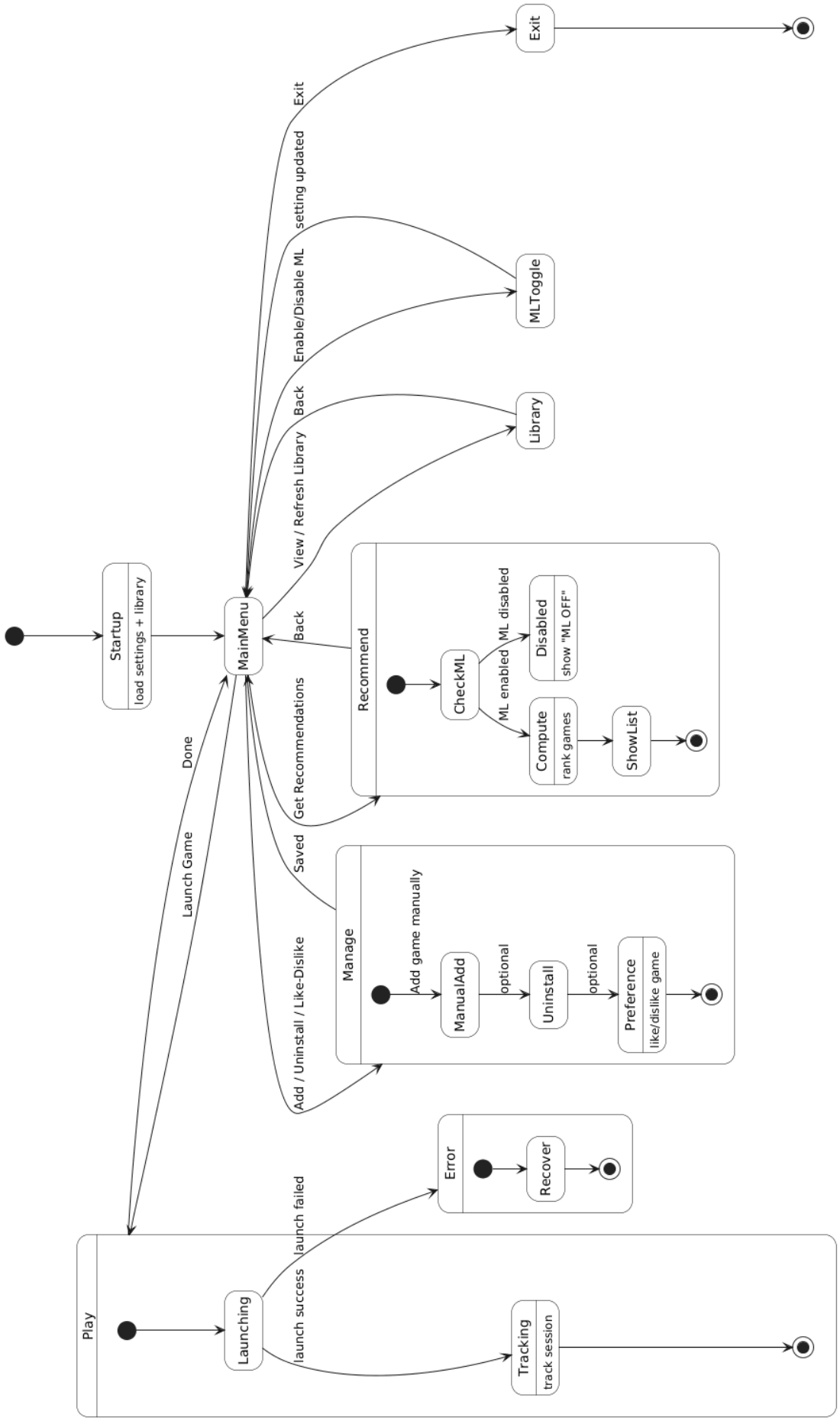

Figure 8: State Diagram (Source: BeTree)

2.8 Assumption and Dependencies

- **Steam Installation**: The Steam client must be installed and accessible on the host machine.
- **Local Binaries**: User must have locally installed game binaries available for detection.
- **Network Connectivity**: Internet access is required for fetching external metadata via the IGDB API.
- **Environment Runtime**: Python runtime and all necessary machine learning libraries must be configured for the recommendation module.
- **System Permissions**: Appropriate administrative or user permissions are required for file system scanning and process initialization.

3. SYSTEM REQUIREMENTS

3.1 User Interface

- The current iteration of the user interface operates via a top-level Command Line Interface (CLI) menu, presenting options for Steam games, local directory games, and an exit function.
- The next iteration of the user interface is currently in development for a more modular and interactive experience.
- System shall display main menu with launcher modes.
- System shall display discovered games with index, title, and identifier/path.
- System shall allow user to select and launch game by input.
- System shall provide clear feedback for invalid input and launch failures.
- System shall display recommendation list with score in ranked order (when ML module invoked).

3.2 Software Interface

- **C++ Core Modules**
  - `game_manager`: local directory scan, sorting, launch.
  - `store_manager`: Steam registry detection, library parsing, Steam launch.
- **Python Module**
  - Metadata ingestion from IGDB.
  - Feature extraction and score calculation.
- **Planned UI Layer**
  - Qt/QML frontend over C++ backend services.

3.3 Database Interface

- **Game catalog records**: Comprehensive data on available titles and assets.
- **Session logs**: Tracking metrics such as start time, end time, and total playtime.
- **User preference flags**: Binary or categorical data (e.g., like/dislike) for personalization.
- **Mood-session records**: Links between specific user moods and their corresponding gameplay sessions.
- **Cached metadata references**: Localized pointers to external assets for optimized performance.

3.4 Protocols

- **Local Process Execution**: Utilizing `ShellExecute` on Windows for native system calls.
- **Steam Launch Protocol**: Integration of the `steam://run/<appid>` URI for direct game initialization.
- **IGDB API Integration**: Secure HTTPS communication using Twitch OAuth for metadata retrieval.

- **Inter-Language Bridge**: Internal C++ ↔ Python data exchange via file-based JSON or an API bridge.

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

- **Game List Retrieval**: Steam and local library fetching must complete within acceptable desktop latency for standard library sizes.
- **Resource Efficiency**: The launcher must avoid excessive memory usage during the scanning and ranking phases.
- **Interactive Performance**: Recommendation generation must complete within a user-tolerable timeframe for fluid usage.

4.2 Security Requirements

- **Secret Management**: No plaintext storage of external API secrets within the source code.
- **Input Validation**: All data from external files or APIs must be validated prior to processing.
- **Sandboxing**: File and process operations are strictly limited to intended game and application paths.
- **Data Privacy**: Local user data must remain on-device unless an explicit export is initiated.

4.3 Software Quality Attributes

- **Reliability**: Graceful handling of invalid paths, malformed manifests, and missing registry keys.
- **Maintainability**: Modular design featuring distinct components for `game_manager`, `steam_manager`, and the ML module.
- **Usability**: A simple, low-friction user flow to ensure an intuitive experience.
- **Portability**: A Windows-first architecture designed with future cross-platform compatibility in mind.

- **Scalability**: Support for expanding game libraries and additional data features over time.

5. OTHER REQUIREMENTS

5.1 Evaluation-Oriented Requirements

- Include requirement traceability mapping (Requirement → Module → Test Case).
- Provide evidence screenshots and logs for scanning, launching, and recommendation output.

5.2 Testing Requirements

- **Unit Tests**: Coverage for parsing and scanning logic.
- **Integration Tests**: Verification of Steam detection and the end-to-end launch flow.
- **Validation Tests**: Evaluation of recommendation quality using precision@k and sanity checks.

5.3 Documentation Requirements

- **User Manual**: Comprehensive instructions for installation and usage.
- **Developer Setup Guide**: Detailed steps for build processes, dependencies, and execution.
- **Project Roadmap**: Documentation of known limitations and planned future work.

A. GLOSSARY

IGDB Internet Game Database; a large structured database of video game information used to programmatically fetch game metadata via its official API.

Launcher An interface or application that opens a game executable upon user input.

One-Hot Encoding A machine learning technique used to convert categorical data (such as user moods or game genres) into a numerical binary format.

Platforms Digital game distribution services (e.g., Steam, GOG, Epic Games Store) where users purchase, subscribe to, and manage games.

TF–IDF Vectorizer Term Frequency – Inverse Document Frequency; a technique used to convert text documents (such as game descriptions) into numerical vectors that machine learning models can process.

B. ANALYSIS MODEL

Data to ML Quantification Pipeline The primary analysis model used to process raw system data into machine-readable formats. This pipeline converts text descriptions into TF–IDF float vectors, playtime into normalized values scaled between 0 and 1, and time-of-day information into temporal encodings.

Engagement Score Calculation Model An analytical model that computes user engagement with a game by calculating a weighted combination of the following components:

- Playtime Score
- Frequency Score
- Length Score

Hybrid Ranker Model The final analysis model responsible for generating recommendations. It calculates a final recommendation score between 0.0 and 1.0 using the following weighted components:

- Preference Match (0.40)
- Similarity to Liked Games (0.35)

- Quality Score (0.15)
- Exploration Factor (0.10)

C. ISSUES LIST

Open Issues

- The machine learning recommendation module currently yields inconsistent recommendation quality. The Python implementation remains a work in progress and requires further calibration.
- The intended Qt/QML user interface design is not yet finalized.
- The system experiences the *Cold Start* data sparsity problem for new users with zero recorded playtime. This is currently mitigated by asking users to manually select preferred genres during the initial setup process.

Resolved Issues

- The project environment previously failed to place `std::filesystem` in a namespace. This issue was resolved by upgrading the project standard from C++14 to C++17.
- The local heuristic game scanner initially detected redundant executables such as crash handlers and uninstallers. This issue was resolved by implementing block parameters for undesired executable names and applying a heuristic that selects the largest executable file.